

Università di Pisa
Master of Science in Informatics for Digital Health
Computing and Networking: Resources and Tools

TremorFSR

Real-time analysis of three force channels and a parallel throughput benchmark

Prof. Marco Danelutto
Academic Year 2025/2026

Ilenia Gulino
Matricola 505852
i.gulino@studenti.unipi.it

Abstract

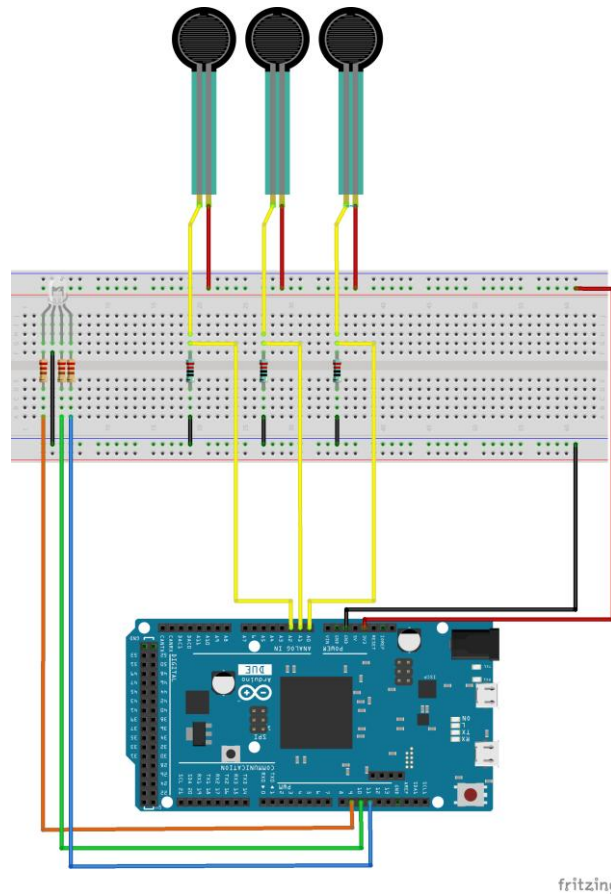
Three force-sensitive resistors, read by an Arduino Due at 500 Hz, are analysed for rhythmic modulation of the applied force within a 4-10 Hz band. The project combines a live sensor application with a parallel throughput study built on the same analysis code. In the live application, acquisition and LED control remain at the edge while completed windows can be analysed either on the acquiring host or on a remote server reached over SSH. In two representative remote sessions no samples were lost, server computation remained below 1.2 ms per window and the median window-to-state latency was about 33 ms, compared with a 512 ms interval between results. The parallel study treats complete recordings as independent session-analysis jobs and compares a FastFlow farm with an OpenMP parallel loop on a 256-core ARM machine. Both implementations scale almost ideally up to 128 workers. At 256 workers FastFlow reaches a speedup of 129 while OpenMP reaches 184 and a separate mapping experiment shows that FastFlow's default thread placement contributes to the saturation at full occupancy.

1. Introduction

The project measures the force applied by three fingers and detects rhythmic modulation within a configurable frequency band, set here to 4-10 Hz. Three force-sensitive resistors (FSRs) are sampled by an Arduino Due at 500 Hz and streamed to a host. The samples are processed in overlapping windows, their spectrum is estimated and the fraction of spectral power inside the selected band is converted into a three-state output. The frequency band, classification thresholds and persistence rule are demonstration parameters, not clinically validated values, so the project makes no claim about tremor or any medical condition. Its focus is the acquisition, communication and computation required to produce the output in real time.

Two performance questions are considered. First, the live application asks whether one sensor stream can be analysed quickly enough to produce a new result every 0.512 s. The same `analyze_window()` function is used in a local configuration and in a distributed edge-to-server configuration. Second, the throughput study asks how many independent recordings can be analysed when many sessions are available at once. The sequential analysis is therefore measured first, a suitable parallel task is selected from the data dependencies, a minimal FastFlow structure is profiled and only the computational bottleneck is replicated. The resulting farm is then compared with an OpenMP parallel for over the same session-level work.

The report first introduces the acquisition chain, from the three FSR sensors to the serial stream produced by the Arduino and then describes the signal-processing steps used to obtain the spectral index and the final state. The live application is presented next, considering both remote execution on the server and local execution on the acquiring host, together with the corresponding timing measurements. The second part of the report focuses on parallel performance. It motivates the choice of a complete session as the unit of parallel work, uses a minimal FastFlow pipeline to identify the computational bottleneck and then compares a replicated FastFlow farm with an OpenMP parallel loop. The experimental setup, workload and measurement procedure are described before presenting the scalability results. The final discussion interprets the observed behaviour in terms of efficiency, full-occupancy effects and thread mapping, while also distinguishing the throughput objective of the benchmark from the latency requirements of the live application. Section 10 concludes the report.



fritzing

Figure 1. The instrument: three sensors as voltage dividers towards A0, A1 and A2, supplied at 3.3 V and the RGB LED on D9, D10 and D11.

2. Acquisition system

Each FSR is a passive analogue sensor whose resistance decreases as the applied force increases. It is connected to an Arduino Due analogue input through a voltage divider supplied at 3.3 V. The board reads the voltage at the junction between the FSR and the fixed resistor through its analogue-to-digital converter.

The three sensors differ in size and sensitivity, so a different fixed resistor is used on each channel. The resistor is chosen close to the sensor resistance in the pressure range of interest, where the divider is most responsive to changes in the FSR. The values used are 200 k Ω on A0, 1 k Ω on A1 and 15 k Ω on A2.

The Arduino samples A0, A1 and A2 every 2 ms, corresponding to 500 samples per second. At each sampling instant it converts the three analogue voltages and transmits one record in the following format:

```
sample_id,time_ms,fsr1,fsr2,fsr3
```

The board performs no signal analysis. Its role is to sample at a regular rate and deliver the measurements to the host. This separation is useful because the spectral analysis assumes uniformly spaced samples, while a dedicated microcontroller can maintain the acquisition schedule independently of the host operating system.

The records are sent through an asynchronous serial connection (UART) exposed over USB as a virtual serial port. The link uses 8N1 framing: one start bit, eight data bits, no parity and one stop bit, so each transmitted byte occupies ten bits on the wire.

The serial rate is 250000 baud. This rate is represented exactly on both sides of the Arduino Due Programming Port connection, considering the SAM3X clock divider and the ATmega16U2 USB-to-serial bridge. With 8N1 framing it provides about 25 kB/s of payload capacity. The application sends roughly 30 characters per sample or about 15 kB/s at 500 Hz, leaving sufficient bandwidth for the stream.

Each record includes a monotonically increasing sample_id and the acquisition timestamp. A discontinuity in the identifier sequence reveals missing samples and the size of the gap gives the number lost. This check matters because a gap would break the uniform spacing assumed by the analysis. The timestamp preserves the original timing information in the recorded CSV data.

3. Signal analysis

The stream is analysed in windows of 4096 samples with a hop of 256. At 500 Hz each window spans 8.192 s and, after the first window is available, a new result is produced every 0.512 s. The complete window provides the data used for one estimate while the smaller hop determines how often that estimate is refreshed.

Mean removal. The static force level is much larger than the rhythmic variation of interest and would dominate the lowest spectral bin. The mean of each analysis window is therefore removed before spectral estimation so that the subsequent computation focuses on changes around the average force level.

Welch spectral estimate. A periodogram computed from a single transform has high statistical variance, so its values can fluctuate even when the underlying signal is similar. Welch's method reduces this variability by splitting the data into overlapping segments, computing one periodogram for each segment and averaging the resulting estimates [1].

Each 4096-sample analysis window is divided into three segments of 2048 samples with a hop of 1024, giving 50 % overlap. A Hann window is applied before each transform to reduce spectral leakage at the segment boundaries [2]. The three periodograms are averaged for each channel. With three channels and three segments per channel, one analysis window therefore requires nine transforms.

The 2048-sample segment length gives a frequency resolution of $500/2048 \approx 0.24$ Hz. The segment length therefore sets the spacing between frequency bins, while the 4096-sample analysis window determines how many overlapping segments contribute to the averaged estimate.

FFT. Computing every discrete Fourier transform coefficient directly would require $O(n^2)$ operations. The implementation instead uses an iterative radix-2 Cooley-Tukey FFT [3], [4]. After a bit-reversal permutation, the algorithm proceeds through $\log_2(n)$ butterfly stages and reduces the transform cost to $O(n \log n)$. For a 2048-sample segment, bin k represents $k \cdot (500/2048)$ Hz and the squared magnitude of the coefficient is used as the spectral power at that frequency.

Band-power index. For each channel, the power of the bins whose centres fall between 4 and 10 Hz is divided by the total spectral power. The final index is the mean of this ratio across the three channels. Normalising by total power reduces sensitivity to the absolute signal amplitude, although real FSR non-linearity and changing contact conditions can still affect the measurement. With 2048-sample segments, the selected band corresponds to bins 17 through 40, with centres from 4.150 to 9.766 Hz.

Decision rule. An index below 0.15 produces NORMAL, a value from 0.15 up to 0.30 produces PRE_ALERT and a value at or above 0.30 produces ALERT only after the threshold has been exceeded in three consecutive windows. This persistence rule limits isolated change. It is also the only stateful part of the analysis: the decision state belongs to one recording session, while different sessions remain independent.

4. Live application

The live application reads the three force channels continuously and produces a new classification whenever a 4096-sample window is complete. The first result is available after 8.192 s and subsequent results are separated by the 256-sample hop or 0.512 s. Acquired samples and per-window results are written to CSV files and the classified state is also shown on the terminal and through the RGB LED.

Remote execution. In the distributed configuration, the Arduino and LED remain on the Windows edge host while the signal analysis runs on a remote server. Each valid serial sample is placed in a bounded queue and a separate writer thread forwards it through an SSH connection. The server accumulates the stream, calls the same `analyze_window()` function used elsewhere in the project and returns one result for each completed window. The edge records that result and updates the LED from the returned state.

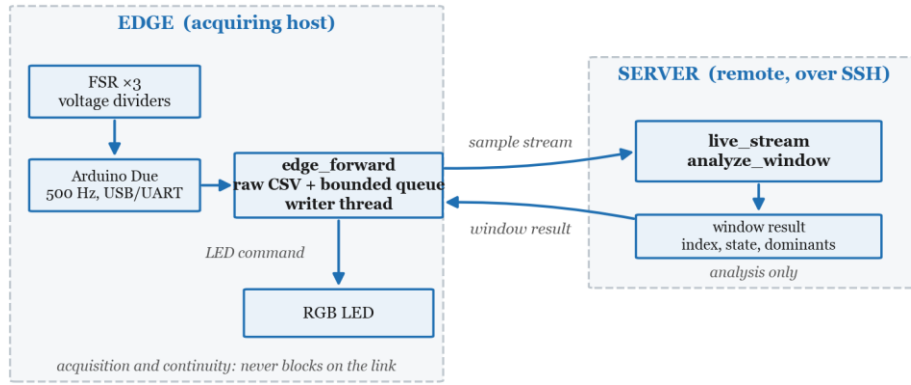


Figure 2. Edge-to-server execution path. Acquisition and continuity checks remain at the edge. A bounded queue decouples serial reading from SSH forwarding, the server performs the analysis and the returned window result drives the RGB LED.

The bounded queue prevents network communication from blocking serial acquisition. A temporary slowdown can therefore delay forwarding without immediately losing sensor data. The queue has a fixed capacity so memory use cannot grow without limit; if it becomes full, new samples are dropped and the event is counted.

Continuity is checked with `sample_id` both before forwarding and in the recorded data. A serial loss or forwarding drop therefore appears as a gap in the identifier sequence. The two sessions reported below contained no gaps and the edge program recorded no forwarding drops, so all analysed windows preserve uniform sampling.

Recorded acquisitions. Two remote acquisitions of about sixty seconds are used as representative live tests: one with steady pressure and one with a voluntary rhythmic modulation. Each contains about 30 000 valid samples and produces 101 analysis windows.

trial	samples	lost	windows	mean index	max index	states
steady pressure	29894	0	101	0.010	0.032	101 NORMAL
rhythmic pressure	29898	0	101	0.275	0.674	32 NORMAL, 36 PRE_ALERT, 33 ALERT

Steady pressure produced only NORMAL windows. In the rhythmic recording the subject first maintained a short steady press and then introduced a voluntary oscillation. The band-power index had an overall mean of 0.275 and reached 0.674, so the classification moved through NORMAL, PRE_ALERT and ALERT and the LED changed from green to yellow and then red.

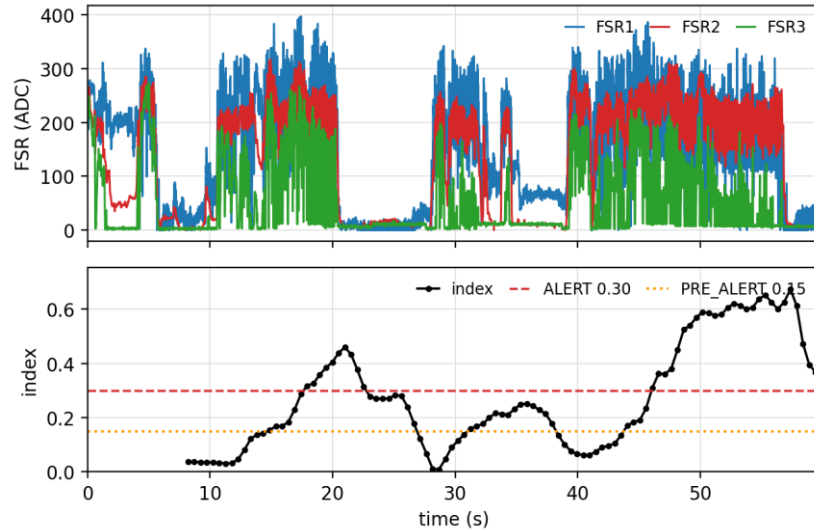


Figure 3. Rhythmic-pressure acquisition analysed on the server. The three force channels are shown above and the band-power index with its PRE_ALERT and ALERT thresholds below.

Timing and latency. Server-side analysis of one completed window required between 0.6 and 1.2 ms, recorded as `processing_us`. This is at most 0.24 % of the 512 ms interval between consecutive results.

The physical response of the live system also depends on windowing: a change in applied force must influence enough samples in the current 8.192 s window before the spectral estimate changes. The additional remote delay can be measured separately once a window is complete.

Window-to-state latency is measured from reception at the edge of a window's last sample to arrival of the corresponding result. Across the two sessions the median was about 33 ms, the 90th percentile about 120 ms and occasional peaks reached about 1.5 s. Since server computation takes only 0.6-1.2 ms, communication and queueing dominate this post-window delay. Temporary stalls can therefore postpone a returned state while acquisition continues; if the bounded queue were to fill, subsequent drops would be counted. No such drop occurred in the reported sessions.

The main timing quantities for the remote configuration are:

quantity	value
sampling period	2 ms
window length	8.192 s
result interval	512 ms
server computation	0.6-1.2 ms
server computation / result interval	$\leq 0.24 \%$
window-to-state latency (median)	~ 33 ms
window-to-state latency / result interval	$\sim 6 \%$

Local execution and comparison. The application can also run entirely on the acquiring host, where serial acquisition, analysis and LED control are handled by the same program. This version is Windows-specific because the serial input uses the Win32 API.

The local configuration calls the same `analyze_window()` function as the server. Processing on the acquiring host remained below 5 ms per window, while the remote server required 0.6-1.2 ms because it runs on different hardware. Those values compare computation only: remote execution additionally incurs the measured communication delay, whereas local window-to-state latency is dominated by the local processing time. Both configurations remain well within the 512 ms result interval under typical conditions.

Both configurations record the same analysis fields, while the remote version adds the measured window-to-state latency. The resulting data can therefore be inspected and plotted with the same analysis tools.

5. Parallelisation strategy

Candidate levels of parallelism. The analysis offers three possible levels of parallel work: transforms within one window, different windows of the same session and complete independent sessions. Their dependencies and usefulness are summarised below.

candidate parallelisation	pattern	chosen	reason
3 channels / 3 Welch segments	data parallel	no	one window already fits the 512 ms live budget by a wide margin
windows of one session	ordered stream / farm	no	the persistence state links consecutive windows
independent sessions	farm	yes	independent, about 0.69 s per task, no resequencing required

Parallelising the nine transforms inside one window would add coordination to a computation that already meets the live timing requirement by a wide margin. Windows of the same session are not independent because the persistence rule carries decision state from one window to the next. Complete sessions avoid both problems: each session has its own state and can be processed without resequencing or shared synchronisation.

Task granularity. Direct measurements confirm that the session also provides a much coarser unit of work than a single window:

one window	595.7 us	
one session	0.689 s	(1156 windows)

A complete session therefore requires about 1150 times more processing time than one window. The parallel task is defined as one session: this preserves the original sequential analysis within each recording while providing independent work that can be assigned directly to different workers.

Minimal pipeline and profiling. Before replication, the session stream is expressed as the smallest FastFlow structure that separates job production, computation and collection:

```
Source -> Compute -> Drain
```

Source emits a pointer to each job, Compute calls `analyze_session()` and Drain receives the completed job. The recordings are loaded before execution and shared read-only, so a task carries a reference to existing input data instead of copying an entire recording. Each job also carries an identifier so its result remains associated with the correct session even if completion order later changes.

The three-stage pipeline is profiled with `ffStats` over twenty-four jobs. FastFlow reports how many tasks each node processes and the total service time accumulated inside its `svc()` method in internal tick units:

ID	stage	tasks	total svc ticks
0	Source	1	5 760
1	Compute	24	578 093 000
2	Drain	24	16 240

Almost all recorded service time belongs to Compute. Source only emits the jobs and Drain only receives them, so replicating either of those stages would not address the bottleneck. The per-task minimum and maximum counters returned invalid overflowed values on this ARM build, therefore only the reliable total svc ticks are reported. The profiling result directly motivates replication of Compute.

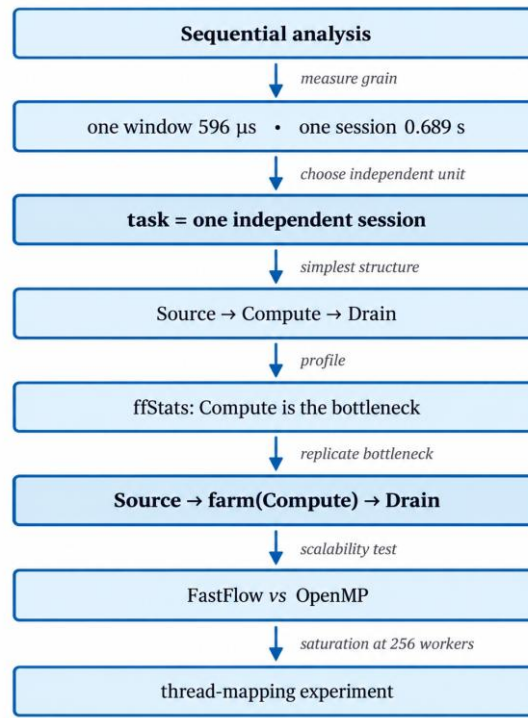


Figure 4. Parallelisation methodology. The sequential analysis is measured first, the independent session task is selected, a minimal Source -> Compute -> Drain stream is profiled and the Compute bottleneck is then replicated before comparison with OpenMP and the final mapping experiment.

6. Parallel implementations

FastFlow farm. Only the Compute stage is replicated:

Source -> farm(Compute) -> Drain

Each worker executes `analyze_session()` on a different session. The jobs cycle through the three synthetic recordings and have similar costs, so FastFlow's default round-robin distribution is sufficient. Output order does not affect correctness because sessions are independent and the job identifier travels with the result.

OpenMP implementation. The OpenMP version applies a parallel for directly to the same in-memory array of session-analysis jobs. It therefore parallelises the same Compute operation without reproducing the FastFlow Source and Drain stages. This gives a direct comparison of worker-level parallelism on the same workload.

Expected scaling. The jobs are independent and similar in cost, so with nw workers the ideal execution time is $T(nw) \approx T_{seq}/nw$ and the ideal speedup is close to nw . At about 0.69 s per task, useful work should dominate runtime overhead through most of the worker range. The widest FastFlow configuration is different: 256 workers are accompanied by emitter, collector, Source and Drain, giving 260 active threads on 256 cores. Some departure from ideal scaling is therefore expected at full occupancy.

7. Benchmark methodology

Synthetic workload. Three fixed synthetic recordings provide known input behaviour. Each channel contains a different constant baseline plus Gaussian noise with a standard deviation of 2 ADC counts. `calm` contains no periodic component, `mild_5hz` adds a 5 Hz sinusoid with amplitude 1.5 ADC counts and `tremor_5hz` uses amplitude 18 ADC counts. Small phase offsets between channels avoid perfect synchrony. The same files are used in every configuration and run.

input	windows	mean index	max index	dominant	states
calm	1156	0.0231	0.0299	131.59 Hz	1156 NORMAL
mild_5hz	1156	0.2336	0.2574	4.88 Hz	1156 PRE_ALERT
tremor_5hz	1156	0.9760	0.9772	4.88 Hz	1154 ALERT, 2 PRE_ALERT

The 5 Hz components appear at 4.88 Hz because the closest FFT bin is separated by about 0.24 Hz. In calm, the reported dominant frequency is simply the largest noise bin and is not used for classification. The band-power index gives the expected states: calm remains NORMAL, mild_5hz remains PRE_ALERT and tremor_5hz reaches ALERT after the first two persistence windows.

Experimental platform. The benchmark ran on a two-socket ARM server with 256 cores, formed by two 128-core Ampere Neoverse-N1 processors with one hardware thread per core and no simultaneous multithreading. The system runs Ubuntu 22.04. The code was compiled with g++ 11.4 using -std=c++20 -O3 -Wall -pthread -fopenmp and the header-only FastFlow library [5]. The main scalability measurements use FastFlow's default thread mapping.

Benchmark workload. The campaign contains 256 independent session-analysis jobs created by cycling through the three synthetic recordings. The files are loaded into memory before timing and remain shared read-only. Each job contains a pointer to its input and storage for its result, so the measured region includes analysis and runtime overhead but no file access. With 256 jobs, one task is available for every worker even at the widest setting.

Measurement procedure. Each configuration is executed seven times. The fastest and slowest runs are discarded and the remaining five are averaged; their retained minimum and maximum are also stored. One untimed sequential pass precedes the campaign to warm the input data and code path. The same procedure is used for the mapping experiments.

Shared environment. The benchmark machine is shared. Its state was recorded around the campaign with uptime and who: two login sessions from another account were present and the load averages before the run were 1.04, 4.90 and 4.51 over 1, 5 and 15 minutes. These values are small relative to 256 cores, although background activity remains a possible source of variation at full occupancy.

Correctness. After every timed run, the 256 summaries are compared with a sequential reference outside the timed region. Floating-point fields use a tolerance of 10^{-12} . Each session is still analysed sequentially by one worker, so the floating-point operations within a session are not reordered and exact agreement was observed in every configuration.

8. Performance results

The sequential baseline for the 256 jobs is 176.8 s, corresponding to 1.45 sessions/s. The single-Compute FastFlow pipeline has the same execution time, so its speedup is 1.00. The absolute times at the three highest worker counts show how the measured executions compare with the ideal T_{seq}/nw time:

workers	ideal time	FastFlow measured	OpenMP measured
64	2.763 s	2.794 s	2.773 s
128	1.381 s	1.421 s	1.391 s
256	0.691 s	1.367 s	0.959 s

Across the complete worker range, the corresponding speedup and parallel efficiency are:

workers	farm speedup	OpenMP speedup	farm eff.	OpenMP eff.
2	2.00	2.00	1.00	1.00
4	4.00	3.99	1.00	1.00
8	8.01	7.99	1.00	1.00
16	15.96	15.99	1.00	1.00
32	31.81	31.94	0.99	1.00
64	63.29	63.76	0.99	1.00
128	124.4	127.1	0.97	0.99
256	129.4	184.5	0.51	0.72

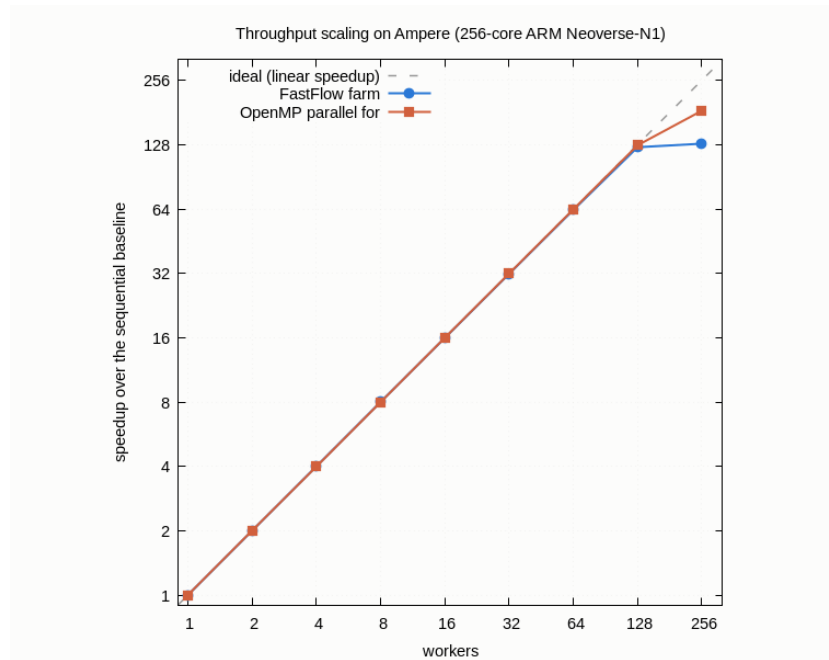


Figure 5. Speedup on the 256-core ARM machine. FastFlow and OpenMP remain close to the ideal line through 128 workers and separate at full occupancy.

Both implementations remain close to ideal scaling up to 128 workers. At 256 workers FastFlow reaches a speedup of 129.4 with efficiency 0.51, while OpenMP reaches 184.5 with efficiency 0.72. Their throughputs are about 187 and 267 sessions/s respectively. Variability is concentrated at the OpenMP 256-thread point: the seven runs range from 0.72 to 1.28 s with a trimmed mean of 0.96 s. A separate warm rerun gives a trimmed mean of 0.99 s and still varies from 1.15 to 0.72 s, so the effect is not explained by cold start alone. The full set of retained measurements is stored in results/benchmark.csv.

9. Performance analysis

Scaling at full occupancy. Through 128 workers the task grain is large enough for useful work to dominate runtime overhead. The FastFlow farm then rises only from about 124× to 129× when the worker count doubles to 256. At that point the 256 workers coexist with emitter, collector, Source and Drain, so the 260 active threads oversubscribe the 256 available cores. This makes full occupancy qualitatively different from the lower worker counts.

FastFlow versus OpenMP. Up to 128 workers their performance is effectively the same because both execute the same coarse independent session tasks. At 256 workers OpenMP is faster because its parallel for operates directly on the job array and does not require the additional FastFlow service threads. The OpenMP version is also shorter, adding about two lines to the sequential code where the FastFlow farm needs roughly forty to build the Source, Compute and Drain stages. FastFlow, in exchange, exposes an explicit stream structure whose stages can be profiled, composed and selectively replicated.

Thread mapping. FastFlow pins threads to cores by default. To test whether placement contributes to the full-occupancy saturation, the farm was repeated at 128 and 256 workers with default pinning, with FastFlow mapping disabled and with core 0 excluded. Each case again uses seven runs with the fastest and slowest discarded:

farm workers	default pinning	default mapping disabled	core 0 excluded
128	1.419	1.415	1.413
256	1.365	1.083	1.173

At 128 workers the three mapping choices are almost identical because many cores remain free. At 256 workers, disabling FastFlow's default mapping reduces the trimmed-mean time from 1.365 s to 1.083 s, recovering much of the gap to OpenMP. Excluding core 0 gives 1.173 s, so on this machine the loss of one usable core outweighs any benefit from reserving it for operating-system activity. The experiment therefore identifies default placement as an important contributor to the farm's full-occupancy behaviour.

Throughput and live latency. The farm does not make one session finish faster internally: a worker still processes that session from its first window to its last with the same sequential analysis. Parallelism instead reduces the time needed to complete many independent sessions. This is a throughput objective and is separate from the live application, where one new window arrives every 512 ms and the sequential per-window computation already meets the timing requirement by a wide margin.

10. Conclusion

The project implements a complete acquisition and analysis path for three force channels sampled at 500 Hz. Each 4096-sample window is analysed with Welch's method and a radix-2 FFT and the fraction of spectral power inside 4-10 Hz is converted into NORMAL, PRE_ALERT or ALERT. In the remote live configuration, acquisition and LED control remain at the edge while the server performs the analysis. The two representative sessions contain no lost samples. Server computation is 0.6-1.2 ms per window and the median post-window state latency is about 33 ms, so the application keeps pace with the 512 ms result interval under typical conditions. The local variant uses the same analysis and remains below 5 ms of processing per window.

For throughput, the natural parallel task is one complete session because the persistence rule links windows inside a session while different sessions are independent. Profiling the initial Source -> Compute -> Drain pipeline shows that Compute dominates service time, so only this stage is replicated. FastFlow and OpenMP scale almost ideally through 128 workers on the 256-core ARM system. At 256 workers FastFlow reaches 129× speedup and OpenMP 184×; the FastFlow farm has 260 active threads and its default mapping contributes to the saturation, as shown by the improvement from 1.365 s to 1.083 s when that mapping is disabled. The live and benchmark parts therefore answer two distinct questions with one analysis implementation: real-time response for a single stream and throughput across many independent sessions.

References

1. P. D. Welch, "The use of the fast Fourier transform for the estimation of power spectra," *IEEE Trans. Audio Electroacoust.* 15(2), 1967, pp. 70-73.
2. Window function (Hann window), Wikipedia, <https://en.wikipedia.org/wiki/Window_function>.
3. J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comp.* 19, 1965, pp. 297-301.
4. Fast Fourier transform, CP-Algorithms, <<https://cp-algorithms.com/algebra/fft.html>>.
5. M. Aldinucci, M. Danelutto, P. Kilpatrick, M. Torquati, FastFlow, <<https://github.com/fastflow/fastflow>>.